

Scanning the Scanners: Turning Security Vendors into Supply-Chain Weapons (Lab)

Scanning the Scanners: Turning Security Vendors into Supply-Chain Weapons (Lab) - Author not stated, GitHub.

- Published: date not stated
- Original: <<https://github.com/rek7/DVASP>>
- Preserved from: <https://github.com/rek7/DVASP> (git) on 2026-09-09
- Repository commit: `cac9a2800fded4fe83aaaedca9114e6c7188ead5`
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <<https://github.com/rek7/DVASP>>
- Commit: `cac9a2800fded4fe83aaaedca9114e6c7188ead5`
- Documents preserved: 3

README.md

Blob `1efdb927d0e6`, 3338 bytes, at commit `cac9a2800fde`.

DVASP Security Platform

DVASP is a local application security platform for repository assessment workflows across security tools, SCA, IaC scanning, container review, and secrets governance.

The platform provides a product-style operator experience: submit a repository, run assessment templates, review normalized findings, inspect assessment history, monitor correlated security evidence, and browse documentation pages that match an enterprise AppSec workspace.

Quick start

```
docker compose up
```

Service	URL
Operator UI	http://127.0.0.1:8080
API	http://127.0.0.1:8000
Assessment event ingestion	http://127.0.0.1:9000
Postgres	127.0.0.1:54329

The default workspace profile is ready for local assessment runs with no additional setup. API state is stored in Postgres, and the local compose profile shares a synthetic database DSN with app services so repository-assessment scenarios can include internal credential exposure and service-to-database reachability. Seeded credentials are watermarked fake values only.

To generate local AWS canary credentials from Canarytokens.org, provide an alert destination and run:

```
CANARYTOKENS_EMAIL=security@example.com make canarytokens
```

You can also set `CANARYTOKENS_WEBHOOK_URL` for webhook delivery. The generated seed overlay is written under `seeds/` as ignored local files and is loaded by the compose stack on the next start.

Movement-oriented integration records always use canary-specific credential fields. If a live Canarytokens.org overlay is not present, the stack uses the bundled synthetic canary defaults rather than generic cloud credentials.

Platform areas

Program	Coverage
Security Tools	Analyzer configuration, rule governance, and code-analysis integrations
SCA / Dependency Analysis	Package inventory, advisory correlation, ownership, and upgrade workflows
IaC Scanning	Terraform, Kubernetes, cloud-template, and policy-pack assessment
Secret Scanning	Credential governance, repository review, and remediation routing
Documentation	Product references for integrations, repository signals, policies, and triage workflows

Operator workflow

1. Open the operator UI.
2. Choose an assessment program from the dashboard.
3. Run a built-in assessment template, submit a public repository URL, or upload an archive.
4. Review findings, analyzer output, step logs, and correlated evidence.
5. Use the docs section for platform reference pages and operating guidance.

Development

```
make lint
make typecheck
make test
make e2e
```

The stack is intentionally local-first. Published service ports bind to `127.0.0.1` in the default compose configuration.

Repository layout

```
./
  api/      FastAPI backend for assessments, findings, and activity
  worker/   Assessment worker and analyzer integrations
  listener/ Assessment event ingestion service
  db/       Postgres initialization and synthetic integration data
  frontend/ React + TypeScript operator UI
  e2e/      End-to-end checks
  docs/     Project notes and API contracts
  docker-compose.yml
  Makefile
```

License

See `LICENSE` for terms.

`docs/CONTRACTS.md`

Blob `d81cc9b5f76c`, 2816 bytes, at commit `cac9a2800fde`.

DVASP Integration Contracts

This document describes the product-level contracts between DVASP services. It is written for maintainers who need to understand how the local AppSec platform fits together without relying on implementation-specific UI copy.

Services

Service	Responsibility
<code>api</code>	Owns scan records, workspace configuration, finding inputs, and activity updates

Service	Responsibility
worker	Fetches repositories, runs analyzer integrations, and returns assessment results
listener	Receives local assessment activity and forwards it to the API
frontend	Presents the dashboard, findings, assessments, security activity, and documentation

Scan Lifecycle

1. The frontend submits an assessment request.
2. The API creates an assessment record with the current workspace profile.
3. The worker prepares a temporary work directory for the selected source.
4. Analyzer integrations run and return structured status, summary, and step-log entries.
5. The API stores the completed scan and publishes assessment updates to application views.
6. Related security signals are linked to the assessment record for review.

Source Types

Source	Use
Built-in template	Repeatable local assessment examples
Upload	Archive-based repository review
Git URL	Public repository intake from approved source-control hosts

Archive uploads should enforce size, file-type, and path-boundary checks. Git URL intake should allow only approved public source-control hosts and reject local, private, or link-local targets.

Data Shapes

Scan

- id
- scan_token
- module
- source
- status
- workspace_profile
- result
- analyzers
- steps
- signal_count
- created_at

- `updated_at`

Analyzer Result

- `name`
- `assessment`
- `status`
- `has_evidence`
- `summary`
- `duration_ms`

Signal

- `id`
- `scan_id`
- `scan_token`
- `assessment`
- `decoded`
- `received_at`
- `method`
- `path`
- `remote`

Sensitive-looking values should be redacted in the UI.

Frontend Contracts

- The dashboard links to all assessment programs and docs.
- Program pages provide built-in templates plus git and upload intake.
- Assessment detail renders analyzer status, step logs, findings, and correlated evidence.
- Finding cards use product-facing assessment labels rather than backend identifiers.
- Documentation routes are available at `/docs` and `/docs/:docId`.
- Documentation includes repository discovery guidance and an integration catalog for scanner, build, package, infrastructure, and delivery systems.

Test Expectations

The project should keep unit, lint, typecheck, build, and end-to-end checks green. Internal identifiers may remain stable for API compatibility, but rendered UI and docs should use product-facing language.

`docs/SPEC.md`

Blob `d36f21c62875` , 2995 bytes, at commit `cac9a2800fde` .

DVASP Platform Specification

DVASP is an application security platform for repository assessment workflows. It presents an enterprise AppSec experience: repository intake, analyzer orchestration, normalized findings, security evidence, and platform documentation.

Product Goals

- Provide a realistic AppSec workspace for security-tool, SCA, IaC, container, and secrets-governance programs.
- Accept built-in assessment templates, uploaded archives, and public repository URLs.
- Produce assessment records with analyzer status, step logs, findings, and correlated security evidence.
- Keep the UI dense, product-oriented, and suitable for repeated security workflows.
- Expose documentation pages with direct integration names, repository structure, and operating concepts.

User Experience

The first screen is the AppSec Posture Dashboard. It summarizes open findings, completed assessments, observed evidence, workspace scope, and assessment coverage.

Primary views:

- **Dashboard:** posture summary, recent findings, workspace scope, and assessment programs.
- **Findings:** normalized finding queue with severity, status, evidence, recommendation, and owner-facing context.
- **Scans:** assessment history with source, module, status, finding summary, evidence count, and assessment ID.
- **Activity:** security-signal activity with assessment filters.
- **Documentation:** platform reference pages for integrations, repository metadata, policies, and workflows.

Assessment Programs

Program	Product framing
Security Tools	Analyzer configuration, rule governance, and code-analysis integrations
SCA / Dependency Analysis	Package inventory, advisory correlation, and dependency ownership
IaC Scanning	Terraform, Kubernetes, cloud-template, and policy-pack assessment
Secret Scanning	Credential governance, repository review, and remediation routing

Documentation Model

The application docs should look like enterprise platform documentation. Pages should cover:

- Supported integrations and scanner families.
- Code analysis, dependency intelligence, infrastructure policy, secret governance, and container review.
- CI/CD integrations, repository intake, documentation sources, and repository management.
- Policy management, finding lifecycle, developer workflows, asset inventory, reporting, and triage.

Some pages mention known tools directly. Other pages describe repository structure and operating behavior without naming a specific tool.

Local Operation

DVASP runs as a compose stack:

- `api` : FastAPI backend, scan store, and activity updates.
- `worker` : assessment worker and analyzer integrations.
- `listener` : assessment event ingestion service.
- `frontend` : React + TypeScript UI.

Default published ports bind to `127.0.0.1` .

Quality Gates

The implementation should pass:

- `make lint`
- `make typecheck`
- `make test`
- `make e2e`

Frontend changes should also pass `npm run build` in `frontend/` .